Sablonul de proiectare Singleton

- Permite restrictionarea numarului de instantieri ale unei clase la un singur obiect
- Utilizare
  - in situatii in care avem obiecte care trebuie accesate din mai multe locuri ale aplicatiei
  - impreuna cu alte pattern-uri: Factory, Builder, Prototype (de exemplu ne dorim un singur obiect factory pentru a crea obiecte de un anumit tip)

Implementare

- In Java pentru a asigura o singura instantiere a clasei:
  - constructorul trebuie sa fie private
  - instanta sa fie oferita printr-o metoda statica, publica (getInstance)

Exemplu de implementare:

```
package singleton;

public class GovernmentLazy {
    private GovernmentLazy() {}
    static private GovernmentLazy instance = null;
    public static GovernmentLazy instance()
    {
        if(instance==null) //Lazy instantiation
            instance= new GovernmentLazy();
        return instance;
    }
```

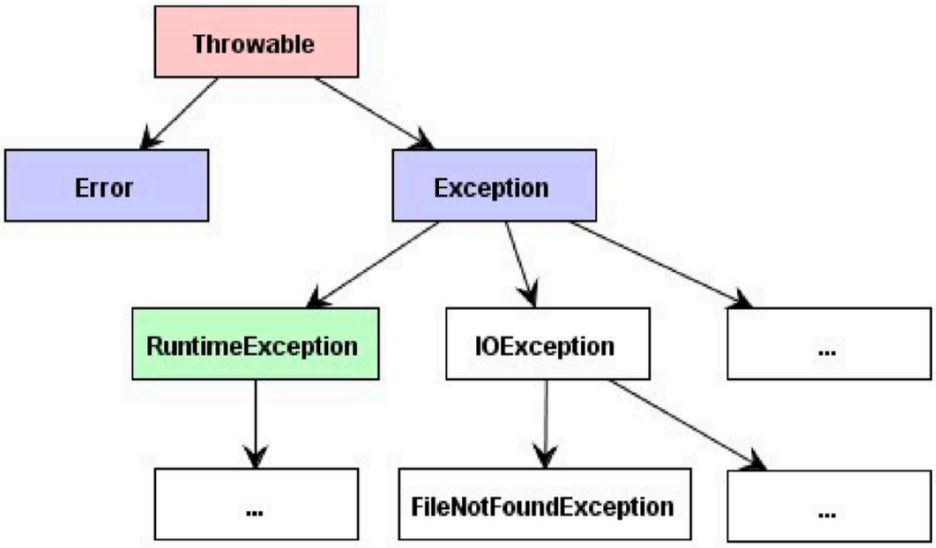
Exceptii

- O exceptie este o situatie "anormala" ce are loc in timpul executiei programului.
- Folosite corect si eficient, exceptiile imbunatatesc atributele de calitate ale sistemelor soft.

Exemplu

```
public static void main(String[] args)
{
    int i=Integer.parseInt("12b");
}
```

Ierarhia claselor ce definesc exceptii



Tipuri de exceptii

- Clasele Error si RunTimeException, impreuna cu descendentii lor, formeaza categoria exceptiilor neverificate, adica exceptii care pot fi generate fara obligativitatea ca ele sa apara in clauzele throws
- Restul exceptiilor sunt verificate, adica la compilare se verifica daca exista clauze throws corespunzatoare.

Tratarea exceptiilor try - catch - finally

```
try {
  // Bloc de instructiuni
  metodaX()
  metodaY()        ──►  S-a generat o excepţie
  metodaZ()
}
catch (TipExceptie1 variabila) {
  // Tratarea exceptiilor de tipul 1
}
catch (TipExceptie2 variabila) {
  // Tratarea exceptiilor de tipul 2
}
catch (TipExceptie3 | TipExceptie4 variabila) {
  // Tratarea exceptiilor de tipul 3 sau 4
}
finally {
  // Cod care se executa indiferent daca apar sau nu exceptii
}
...execuţia continuă
```

Avantajele folosirii exceptiilor

1. Separarea codului pentru tratarea unei erori de codul in care ea poata sa apara.
2. Gruparea erorilor dupa tipul lor.
3. Propagarea unei exceptii pana la un analizator de exceptii corespunzator.

1. Separarea codului

```
int openResult = open();

if (openResult == FILE_NOT_FOUND) {
    // handle error
} else if (openResult == INUFFICIENT_PERMISSIONS) {
    // handle error
} else {// SUCCESS
    int readResult = read();
    if (readResult == DISK_ERROR) {
        // handle error
    } else {
        // SUCCESS
     ...
    }
}
```

```
try {
    open();
    read();
     ...
} catch (FILE_NOT_FOUND) {
    // handle error
} catch (INUFFICIENT_PERMISSIONS) {
    // handle error
} catch (DISK_ERROR) {
    // handle error
}
```

2. Gruparea erorilor dupa tipul lor

```
try {
    String driverName = new String(Files.readAllBytes(Paths.get("driver.txt")));
    Class.forName(driverName).newInstance();
} catch (IOException ex) {
    // probleme cu fisierul din care vrem sa citim
} catch (ClassNotFoundException ex) {
    // nu exista clasa driver
} catch (IllegalAccessException ex) {
    // lipsa acces clasa
} catch (InstantiationException ex) {
    // clasa nu poate fi instantiata
}
```

3. Propagarea exceptiilor

```
int metoda3() throws TipExceptie {
      ...
      throw new TipExceptie();
      ...
}
int metoda2() throws TipExceptie {
      ...
      metoda3();
      ...
}
int metoda1() {
      try {
          metoda2();
      } catch (TipExceptie e) {
          //proceseaza exceptie
      }
  }
```

Definirea propriilor clase de exceptii

```
public class ExceptieProprie extends RuntimeException {
    //Proprietati si constructori
    public ExceptieProprie(String mesaj) {
        super(mesaj); // Apeleaza constructorul superclasei }
    }
}
```

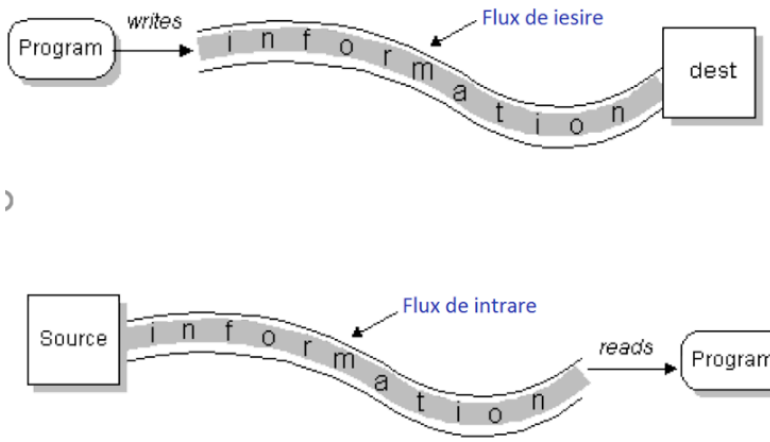
```
public class ValidatorException extends Exception {
   public ValidatorException(String message) { super(message); }
}
```

```
public class StudentValidator implements Validator<Student> {
   @Override
   public void validate(Student e) throws ValidatorException {
      String errMsg="";
      if (e.getId() == null || "".equals(e.getId()))
         errMsg+="Id error ";
      if (e.getFirstName() == null || "".equals(e.getFirstName()))
         errMsg+="first name error ";
      if (e.getLastName() == null || "".equals(e.getLastName()))
         errMsg+="last name error ";
      if (e.getEmail() == null || "".equals(e.getEmail()))
         errMsg+="email error error ";
      if (errMsg!="")
         throw new ValidatorException(errMsg);
   }
}
```

- Metodele suprascrise pot arunca numai exceptiile specificate de metoda din clasa de baza sau exceptii derivate din acestea.

Conceptul de stream (flux de date)

- Stream = orice sursa sau consumator de date care este capabil sa produca sau sa primeasca unitati de date.

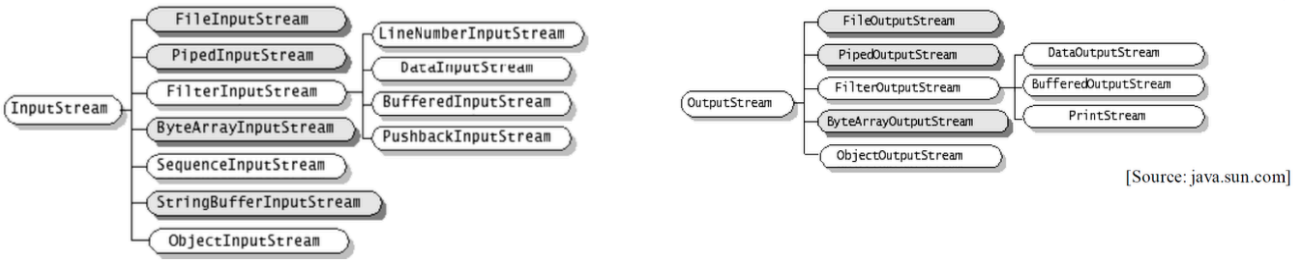


Tipuri de fluxuri de date

- Pe octeti (byte) - unitatea de informatie este octetul
- Pe caractere - unitatea de informatie este caracterul

Fluxuri pe octeti

InputStream, OutputStream - Citire/Scriere octet cu octet



Input Stream

- Clasa abstracta InputStream se situeaza in varful ierarhiei de clase care descriu fluxuri octet sursa.
- Toate clasele derivate implementeaza metoda read()

abstract int	**read()** Reads the next byte of data from the input stream.
int	**read(byte[] b)** Reads some number of bytes from the input stream and stores them into the buffer array b.
int	**read(byte[] b, int off, int len)** Reads up to len bytes of data from the input stream into an array of bytes.
void	**reset()** Repositions this stream to the position at the time the mark method was last called on this input stream.
long	**skip(long n)** Skips over and discards n bytes of data from this input stream.

Output Stream

- Clasa OutputStream se situeaza in varful ierarhiei de clase care descriu fluxuri octet destinatie.
- Pentru fiecare clasa InputStream exista o clasa omolog OutputStream.
- Operatiile din clasa OutputStream sunt operatiile in oglinda ale celor din clasa InputStream.

void	**close()** Closes this output stream and releases any system resources associated with this stream.
void	**flush()** Flushes this output stream and forces any buffered output bytes to be written out.
void	**write(byte[] b)** Writes b.length bytes from the specified byte array to this output stream.
void	**write(byte[] b, int off, int len)** Writes len bytes from the specified byte array starting at offset off to this output stream.
abstract void	**write(int b)** Writes the specified byte to this output stream.

Exemplu - FileInputStream/FileOutputStream

```java
public static void readWithFileInputStream() {
    FileInputStream in = null;
    FileOutputStream out = null;
    try {
        in = new FileInputStream("Fis1.txt");
        out = new FileOutputStream("Fis2.txt");
        int c;
        while ((c = in.read()) != -1) {
            out.write(c);
        }
    } catch (IOException e) {
          e.printStackTrace();
    } finally {
        if (in != null)
            try {
                in.close();
            } catch (IOException e) {
              e.printStackTrace();;}
        if (out != null)
            try {
                out.close();
            } catch (IOException e)
              e.printStackTrace(); }
    }
}
```

```java
public static void readWithFileInputStream(){

    try(FileInputStream is=new
FileInputStream("./data/Messages");
        FileOutputStream os =new
FileOutputStream("./data/CopyMessages")){
        int i;
        while((i=is.read())!=-1){
            os.write(i);
        }
    } catch (FileNotFoundException e) {
        e.printStackTrace();
    } catch (IOException e) {
        e.printStackTrace();
    }
}
```

Try - With - Resources

```java
public static void readBytes_usingTryWithResources() {
    try (FileInputStream in = new FileInputStream("Fis1.txt");
         FileOutputStream out = new FileOutputStream("Fis2.txt"))
    {
        int c;
        while ((c = in.read()) != -1) {
            out.write(c);
        }
    } catch (IOException e) {
        // aici nu avem acces la var in sau out
    }
}
```

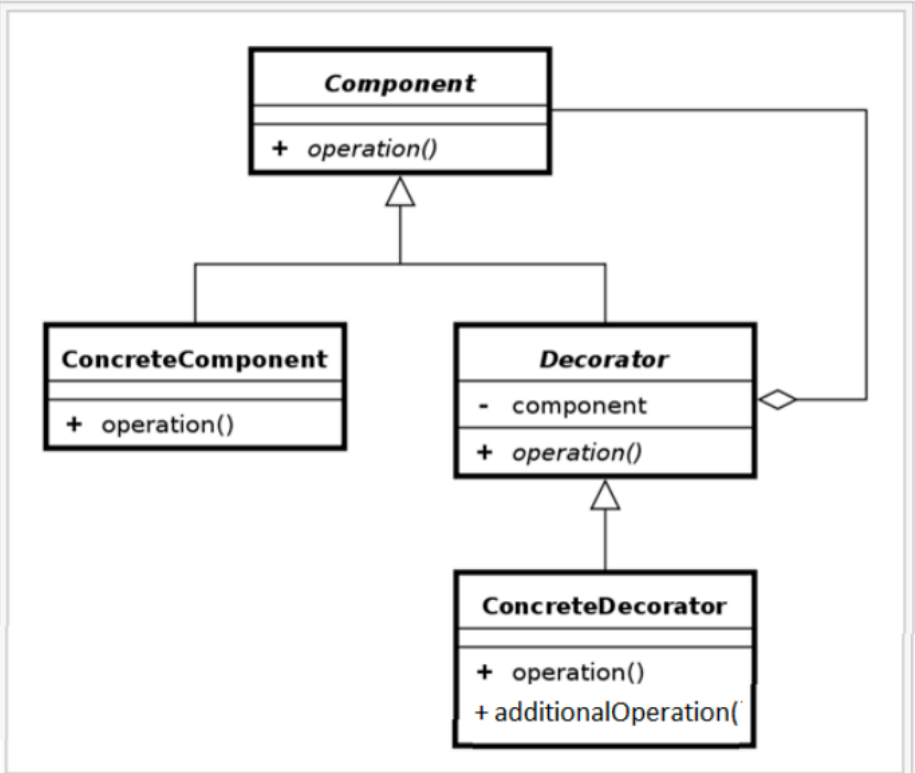
Pentru a folosi *Try-With-Resources* Resursa trebuie sa implementeze interfața java.lang.AutoCloseable

Old stye

```java
public static void readBytes() {
    FileInputStream in = null; FileOutputStream out = null;
    try {
        in = new FileInputStream("Fis1.txt");
        out = new FileOutputStream("Fis2.txt");
        int c;
        while ((c = in.read()) != -1) {
            out.write(c);
        }
    } catch (IOException e) { e.printStackTrace();
    } finally {
        if (in != null)
            try {
                in.close();
            } catch (IOException e) { e.printStackTrace();}
        if (out != null)
            try {
                out.close();
            } catch (IOException e) e.printStackTrace(); }
    }
}
```

Clase decorator pentru fluxuri

- Problema este ca ne dorim sa citim int, string, double...
- De exemplu, avem nevoie de un InputStream care citeste 4 octeti deodata...
- Solutia este **decoratorul design pattern**, acest pattern permite "imbracarea" unui obiect intr-un altul care adauga comportamente suplimentare. Ambele obiecte, cel de baza si decoratorul respecta aceeasi interfata, deci pot fi folosite interschimbabil.



Decorator UML class diagram

Clase decorator pentru fluxuri

- Decoratori:
  - Clasele DataInputStream si DataOutputStream - citirea si scrierea tipurilor primitive
- Adauga metodele :
  - readByte, readInt, readFloat, readBoolean si omoloagele lor, writeByyte, writeInt, etc.

Decoratori - Data Output/Input Stream

```
public static void writeFileDataOutputStream()
{
    try  (DataOutputStream out = new DataOutputStream(new FileOutputStream("fis.meu"))){
        out.writeFloat(10.0f);
        out.writeChar('\n');
        out.writeInt(5);
        out.writeChar('\n');
        out.writeUTF("Hello");

    } catch(IOException e) {e.printStackTrace(); }
}

  public static void readFileDataInputtStream(){
      try (DataInputStream in = new DataInputStream(new FileInputStream("fis.meu"))){
          System.out.println("Citesc un float: "  + in.readFloat());
          System.out.println("Citesc enter: "  + in.readChar());
          System.out.println("Citesc un int: "    + in.readInt());
          System.out.println("Citesc enter: "  + in.readChar());
          System.out.println("Citesc un string: " + in.readUTF());
      } catch(IOException e) { e.printStackTrace();}
  }
```

Decoratori - BufferedOutputStream/BufferedInputStream

```
public static void writeBufferedFileDataInputtStream()
{
    try (BufferedOutputStream bout=new BufferedOutputStream(new FileOutputStream("fis.meu"));
            DataOutputStream out = new DataOutputStream(bout))  {
        out.writeFloat(10.0f);
        out.writeChar('\n');
        out.writeInt(5);
        out.writeChar('\n');
        out.writeUTF("Hello");
    } catch(IOException e) {e.printStackTrace(); }
}
public static void readBufferedDataOutputStream(){
    try (BufferedInputStream bin=new BufferedInputStream(new FileInputStream("fis.meu"));
         DataInputStream in = new DataInputStream(bin))
    {
        System.out.println("Citesc un float: "  + in.readFloat());
        System.out.println("Citesc enter: "  + in.readChar());
        System.out.println("Citesc un int: "    + in.readInt());
        System.out.println("Citesc enter: "  + in.readChar());
        System.out.println("Citesc un string: " + in.readUTF());
    } catch(IOException  e) { e.printStackTrace();}
}
```

Streamuri standard

- System.in de tipul InputStream
- System.out de tipul PrintStream
- System.err de tipul PrintStream

Fluxuri pe caractere

- Odata cu introducerea caracterelor Unicode a crescut numarul de octeti necesari pentru a reprezenta un caracter, de la 1 la 2.
- De asemenea, a aparut notiunea de codificare a sirurilor (encoding).
- In varful ierarhiilor claselor care lucreaza cu caractere se afla **Reader** si **Writer**.